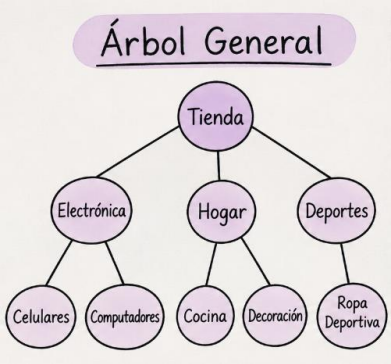
1. ¿Qué son los árboles generales?

Un árbol general es una estructura de datos no lineal que permite organizar información de manera jerárquica. Se caracteriza porque cada nodo puede tener cualquier cantidad de hijos, es decir, no tiene un límite máximo establecido de descendientes directos.

Su estructura comienza con un nodo principal llamado raíz, del cual pueden salir diferentes ramas que conectan con otros nodos. Estos nodos también pueden tener sus propios hijos, creando diferentes niveles dentro del árbol.

Por ejemplo, podemos imaginar la estructura de una empresa. En la parte superior se encuentra el gerente general, quien puede tener varios gerentes de área. Cada gerente puede tener diferentes empleados a su cargo. Esta organización representa un árbol general porque cada nodo puede tener una cantidad diferente de hijos.



Características de los árboles generales

- Estructura jerárquica: los elementos se organizan por niveles, comenzando desde una raíz principal.
- Cantidad de hijos variable: cada nodo puede tener cero, uno o muchos hijos.
- Nodo raíz: es el elemento principal del árbol y no tiene un padre.
- Nodos internos: son aquellos que tienen al menos un hijo.
- Nodos hoja: son los elementos que no tienen hijos y representan el final de una rama.
- Relación entre nodos: cada nodo, excepto la raíz, tiene un único padre.
- Conexiones: los nodos se comunican mediante ramas o enlaces que permiten establecer las relaciones jerárquicas.

Elementos que componen un árbol general

- Raíz: es el nodo inicial y principal de toda la estructura.
- Padre: es el nodo que tiene uno o varios hijos.
- Hijo: es un nodo que depende directamente de otro.
- Hermanos: son los nodos que comparten el mismo padre.
- Hojas: son los nodos que no tienen descendientes.
- Ramas: son las conexiones que unen los diferentes nodos.
- Subárbol: es una parte del árbol que comienza en un nodo y contiene todos sus descendientes.
- Altura: representa la longitud del camino más largo desde la raíz hasta una hoja.

Aplicaciones de los árboles generales

- Sistemas de archivos: permiten organizar carpetas, subcarpetas y documentos dentro de un computador.
- Árboles genealógicos: representan las relaciones entre diferentes integrantes de una familia.
- Organigramas empresariales: permiten visualizar los cargos y las dependencias dentro de una organización.
- Páginas web: representan estructuras jerárquicas de elementos HTML.
- Bases de datos: permiten representar información que tiene relaciones jerárquicas.
- Sistemas de navegación: ayudan a organizar menús y opciones dentro de aplicaciones.

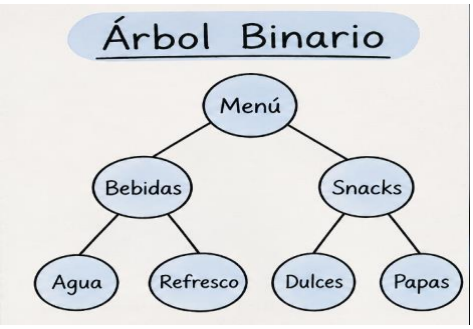
2. ¿Qué son los árboles binarios?

Un árbol binario es un tipo especial de árbol que se caracteriza porque cada nodo puede tener como máximo dos hijos. Estos hijos reciben los nombres de hijo izquierdo e hijo derecho.

Esto significa que cada nodo puede tener cero, uno o dos hijos, pero nunca más de dos. A diferencia de los árboles generales, donde un nodo puede tener muchos descendientes directos, los árboles binarios tienen una estructura más limitada y definida.

Los árboles binarios comienzan con una raíz principal, de la cual pueden salir dos ramas: una hacia la izquierda y otra hacia la derecha. Cada uno de estos hijos puede tener también dos hijos, formando nuevos niveles dentro de la estructura.

Un ejemplo sencillo es un sistema de preguntas donde cada decisión tiene dos posibles respuestas, como sí o no. Dependiendo de la respuesta, el programa puede continuar por una rama diferente.



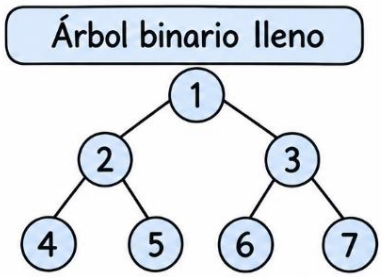
Características de los árboles binarios

- Máximo dos hijos: cada nodo puede tener como máximo dos descendientes directos.
- Hijo izquierdo: es el nodo que se encuentra conectado por la rama izquierda.
- Hijo derecho: es el nodo que se encuentra conectado por la rama derecha.
- Raíz: es el primer nodo de la estructura.
- Nodos hoja: son aquellos que no tienen hijos.
- Organización jerárquica: los elementos se distribuyen en diferentes niveles.
- Recorridos: permiten visitar los nodos en diferentes órdenes para procesar la información.
- Búsqueda: algunos tipos de árboles binarios permiten encontrar información de manera eficiente.

Tipos de árboles binarios

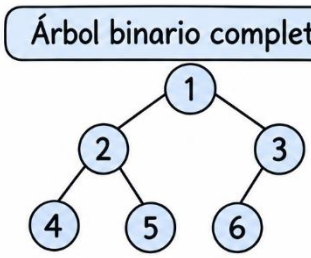
2.1 Árbol binario lleno

Es aquel en el que cada nodo tiene exactamente dos hijos o ninguno. No existen nodos que tengan solamente un hijo.



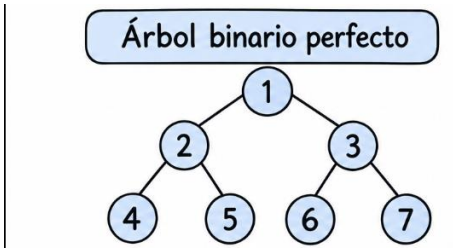
2.2 Árbol binario completo

Es aquel en el que todos los niveles están completamente llenos, excepto posiblemente el último nivel. En este último, los nodos se colocan de izquierda a derecha sin dejar espacios intermedios.



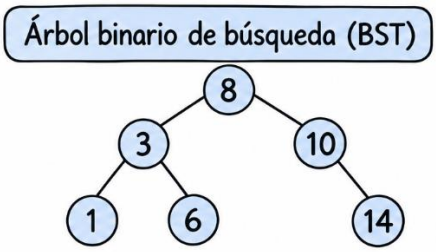
2.3 Árbol binario perfecto

Es aquel en el que todos los niveles están completamente llenos y todas las hojas se encuentran a la misma profundidad. Cada nodo interno tiene exactamente dos hijos.



2.4 Árbol binario de búsqueda (BST)

Es un tipo de árbol binario que organiza los valores siguiendo una regla: los valores menores que un nodo se ubican a su izquierda y los valores mayores se ubican a su derecha. Esta organización facilita las operaciones de búsqueda, inserción y eliminación de datos.



Recorridos de los árboles binarios

Una de las características más importantes de los árboles binarios es que permiten recorrer todos sus nodos siguiendo diferentes órdenes. Los principales recorridos son:

- Preorden: primero se visita la raíz, después el subárbol izquierdo y finalmente el derecho.
- Inorden: primero se visita el subárbol izquierdo, después la raíz y finalmente el derecho.
- Postorden: primero se visita el subárbol izquierdo, luego el derecho y finalmente la raíz.
- Por niveles: se visitan los nodos comenzando por la raíz y avanzando nivel por nivel, de izquierda a derecha.

Estos recorridos son importantes porque permiten consultar, modificar y procesar la información almacenada en el árbol.

Aplicaciones de los árboles binarios

- Motores de búsqueda: ayudan a organizar y localizar información.
- Bases de datos: permiten implementar estructuras de búsqueda y organización.
- Compiladores: se utilizan para representar expresiones y operaciones matemáticas.
- Inteligencia artificial: permiten representar decisiones y diferentes posibilidades.
- Ordenamiento de datos: ayudan a clasificar información según determinados criterios.
- Sistemas de búsqueda: permiten encontrar valores dentro de conjuntos de datos.

4. Diferencias entre árboles generales y árboles binarios

Aunque los dos tipos de árboles permiten organizar información de forma jerárquica, existen diferencias importantes en su estructura y funcionamiento.

Característica	**Árbol general**	**Árbol binario**
Cantidad de hijos	No tiene límite fijo	Máximo dos
Organización	Jerárquica	Jerárquica
Nodo raíz	Sí	Sí
Hijos	Puede tener varios	Izquierdo y derecho
Nodos hoja	Sí	Sí
Recorridos	Por niveles y otros métodos	Preorden, inorden, postorden y por niveles
Uso principal	Representar jerarquías	Búsqueda y organización de datos

4. Ventajas y desventajas

Árboles generales

Ventajas:

- Permiten representar estructuras complejas y jerarquías de diferentes tamaños.
- No limitan la cantidad de hijos de cada nodo.
- Son útiles para organizar información que tiene muchas relaciones.

- Facilitan la representación de estructuras como carpetas y organigramas.

Desventajas:

- Su implementación puede ser más compleja dependiendo de la cantidad de hijos.
- La búsqueda de información puede requerir recorrer muchos nodos.
- Pueden necesitar mecanismos adicionales para administrar sus diferentes conexiones.

Árboles binarios

Ventajas:

- Su estructura es relativamente sencilla de implementar.
- Permiten realizar búsquedas, inserciones y eliminaciones.
- Facilitan la organización de información mediante dos ramas.
- Los árboles binarios de búsqueda equilibrados pueden realizar operaciones de búsqueda en tiempo logarítmico.

Desventajas:

- Cada nodo tiene un límite de dos hijos.
- Si un árbol binario de búsqueda queda muy desequilibrado, las búsquedas pueden volverse lentas.
- Requieren una organización adecuada para aprovechar su eficiencia.
- Dependiendo de su implementación, pueden necesitar memoria adicional para almacenar las conexiones entre nodos.

5. Importancia de los árboles en las estructuras de datos

Los árboles son fundamentales en la programación porque permiten solucionar problemas relacionados con la organización, almacenamiento y búsqueda de información.

A diferencia de las estructuras lineales, los árboles permiten representar relaciones entre diferentes elementos sin necesidad de organizarlos en una sola secuencia. Esto facilita el manejo de grandes cantidades de información y permite desarrollar algoritmos que aprovechan las relaciones entre los datos.

Los árboles generales son especialmente útiles cuando se necesita representar estructuras jerárquicas con diferentes cantidades de elementos dependientes. Por otro lado, los árboles binarios son importantes cuando se requiere organizar información mediante dos ramas y realizar operaciones de búsqueda y clasificación.

6. CODIGO

```
#include <iostream>
using namespace std;

// Estructura del nodo
struct Nodo {
    int dato;
    Nodo* izquierda;
    Nodo* derecha;
```

```
};

// Crear un nuevo nodo
Nodo* crearNodo(int valor) {
    Nodo* nuevo = new Nodo();
    nuevo->dato = valor;
    nuevo->izquierda = NULL;
    nuevo->derecha = NULL;
    return nuevo;
}

// Recorrido PREORDEN
void preorden(Nodo* raiz) {
    if (raiz != NULL) {
        cout << raiz->dato << " ";
        preorden(raiz->izquierda);
        preorden(raiz->derecha);
    }
}

// Recorrido INORDEN
void inorden(Nodo* raiz) {
    if (raiz != NULL) {
        inorden(raiz->izquierda);
        cout << raiz->dato << " ";
        inorden(raiz->derecha);
    }
}

// Recorrido POSORDEN
void posorden(Nodo* raiz) {
    if (raiz != NULL) {
        posorden(raiz->izquierda);
        posorden(raiz->derecha);
        cout << raiz->dato << " ";
    }
}

int main() {

    // Crear el árbol binario
    Nodo* raiz = crearNodo(1);

    raiz->izquierda = crearNodo(2);
    raiz->derecha = crearNodo(3);

    raiz->izquierda->izquierda = crearNodo(4);
    raiz->izquierda->derecha = crearNodo(5);

    raiz->derecha->izquierda = crearNodo(6);
    raiz->derecha->derecha = crearNodo(7);

    // Mostrar recorridos
    cout << "RECORRIDO PREORDEN: ";
```

```
    preorden(raiz);

    cout << "\nRECORRIDO INORDEN: ";
    inorden(raiz);

    cout << "\nRECORRIDO POSORDEN: ";
    posorden(raiz);

    return 0;
}
```